

# Data Structure

Please comment with your name at the top of the code. Your program should be tested before submitting; make sure it actually compiles and works! Please submit your project through replit.

## Program Description

You need to write a C++ program to implement a variable length 2-dimension array of integer. You can think about a 2-dimension array is an array of arrays as illustrated in Figure 1. I want you to **dynamically allocate** the memory for the **second dimension arrays** (for each row).

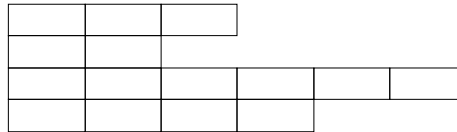


Figure 1: Example of a 2-dimension array.

Your program should allow the user to query the elements of this data structure by providing two values (indices of the arrays) where the first number indicates the row and the second number indicates the column. Also, your program should prompt the user if the input is out of bound. For instance, if the structure has 3 rows, the user input of 3 should return an out of bound error (remember that the indices start from 0 and hence, 3 rows would have indices 0, 1, and 2). The same boundary check should be applied for each row. For instance, if a row has size 4, the user input 6 should have out of bound prompt for user.

It is a good practice to print out the memory addresses to better understand the memory allocation.

## Input

To build such a 2-dimension array, you need to get the dimension and input information from a file named “**input.txt**”. The “**input.txt**” file has the following format, and generates a 2-dimension array as shown in Figure 2:

4  
 3  
 5 9 12  
 2  
 11 7  
 6  
 1 3 9 0 12 2  
 4  
 21 8 8 16

|    |   |    |    |    |   |
|----|---|----|----|----|---|
| 5  | 9 | 12 |    |    |   |
| 11 | 7 |    |    |    |   |
| 1  | 3 | 9  | 0  | 12 | 2 |
| 21 | 8 | 8  | 16 |    |   |

Figure 2: Example of a 2-dimension array.

The first number in the input indicates the number of rows (number of horizontal arrays that should be defined dynamically). After that the first row, you will have a row of one number indicating the size of each row following by the element of that row.

## Output

When the structure is populated, you should allow the user to query the elements. In the above example, query 0 0 should return 5 and query 3 1 should return 8. Meanwhile, queries 3 4 and 4 1 should prompt out of bound. The program should terminate with the user input  $-1 -1$ . Be careful to release the allocated memory.